Объектно-ориентированное программирование в python

Класс

Python сильно более объектно ориентирован, чем многие языки. Почти все что вы встретите в python будет объектом.
Функции, числа, классы, модули - это все объекты.

По сути тип данных в python - это класс объекта. Так что для получения класса объекта есть `type`

```python
type(1)
```

```
<class 'int'>
```

```python
type([1, "2", (3,)])
```

```
<class 'list'>
```

```python
class A: ...
type(A())
```

```
<class '__main__.A'>
```

Но в python как говорилось раньше практически все - это объект К примеру:

```python
type(A)
```

```
<class 'type'>
```

```python
type(type)
```

```
<class 'type'>
```

```python
def func(): ...
type(func)
```

```
<class 'function'>
```

```python
type(type(func))
```

```
<class 'type'>
```

Как мы знаем у классов есть атрибуты и методы.

На уровне реализации языка внутри каждый объект - это struct в Си, которая содержит:

- Счетчик ссылок: `ob_refcnt`
- Указатель на тип: `ob_type`

И выглядит примерно так:

```c
typedef struct _object {
    _PyObject_HEAD_EXTRA
    Py_ssize_t ob_refcnt;
```

```
4     PyTypeObject *ob_type;
5 } PyObject;
```

Тут `_PyObject_HEAD_EXTRA` - макрос, который добавляет указатели на предыдущий и следующий объект в памяти. Это позволяет сборщику мусора ходить по двухсвязному списку всех активных объектов и удалять их в случае необходимости

Для объектов переменной длины так же имеются поле `ob_size`. Такие объекты называются `PyVarObject`.

```c
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_size;
} PyVarObject;
```

Класс в python

Класс создается с помощью ключевого слова `class`:

```python
class Test:
    pass
```

Хотя он и кажется пустым, тем не менее он имеет целую охапку полей и методов, которые python создает автоматически. Их можно увидеть с помощью функции `dir()` и `var()`:

```python
dir(Test)
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__', '__forma
```

```python
vars(Test)
```

```
mappingproxy({'__module__': '__main__', '__firstlineno__': 1, '__static_attributes__': (), '__dict__
```

Чтобы создать объект класса достаточно вызвать его с синтаксисом похожим на функцию

```python
a = Test()
a
```

```
<__main__.Test object at 0x10100af90>
```

На самом деле это и является неявно вызываемыми функциями, которая вызывается каждый раз при создании объекта. А конкретно метод `__new__` класса Test (выделяет память и создает объект данного класса). А так же метод `__init__()` объекта, который дает логику внутрь объекта (к примеру заполняет его поля). Так что семантически такой вариант будет аналогичным:

```python
a = Test.__new__(Test)

if isinstance(a, Test):
    a.__init__()

a
```

```
<__main__.Test object at 0x10109a210>
```

Так что если при написании класса мы определим метод `__init__`, тогда объект класса будет иметь соответсвующую логику::

```python
class Test:
    def __init__(self):
        print("Hello, world!")
        self.a = 123
```

```python
a = Test()
```

```
Hello, world!
```

```python
a.a
```

```
123
```

Аналогично:

```python
b = Test.__new__(Test)

if isinstance(a, Test):
    b.__init__()
```

```
Hello, world!
```

```python

b.a
```

```
123
```

Так же мы можем в рантайме изменять, добавлять и удалять атрибуты и методы

```python
b.b = "Новый аттрибут"
b.b
```

```
'Новый аттрибут'
```

```python


def func(num):
    print(f"Новый метод №{num}")

b.c = func
b.c(1)
```

```
Новый метод №1
```

При этом вызвать новый метод будет именно методом конкретного объекта:

```python
a.b
a.c(2)
```

Такой код вызывает ошибку.

Такой подход позволяет реализовывать активную интроспекцию и рефлексию, меняя, добавляя и удаляя поля в рантайме.

Наследование

По большому счету принцип наследования можно свести к дополнительному способу доступа к атрибутам.

В python наследование организуется через добавление класса родителя в `__mro__` и `__bases__` Если кратко, то `__bases__` хранит в себе классы своих родителей, а `__mro__` линейный список всех своих предков.

К примеру:

```python
class A: ...
class B: ...
class C(A, B): ...

C.__bases__
```

```
(<class '__main__.A'>, <class '__main__.B'>)
```

```python
C.__mro__
```

```
(<class '__main__.C'>, <class '__main__.A'>, <class '__main__.B'>, <class 'object'>)
```

А так же в python существует метод , благодаря которому можно добавить в качестве базового класса объект.
Так же в python в качестве предка можно использовать любой объект с помощью метода `__mro_entries__`. Он принимает `tuple` из всех базовых классов наследника и возвращает `tuple` из классов (то есть экземпляров `type`)

К примеру отнаследуемся от функции:

```python
def my_func():
    print('called')
my_func.__mro_entries__ = lambda bases: (str,)

class MyStr(my_func): ...

print(MyStr.__mro__)
```

```
(<class '__main__.MyStr'>, <class 'str'>, <class 'object'>)
```

```python
print(MyStr('abc'))
```

```
abc
```

И нам на самом деле ничего не мешает изменить, дополнить, удалить предка наследуемого класса

MRO

Выше был упомянут атрибут `__mro__`. Но что это такое?

Порядок разрешения методов (method resolution order) позволяет Python понять, из какого класса-предка нужно вызывать метод, если он не обнаружен непосредственно в классе-потомке. Если у каждого потомка есть только один предок, то задача тривиальна. Происходит восходящий поиск по всей иерархии. Если же используется множественное наследование, то можно столкнуться с определенными проблемами, которые будут рассмотрены позже.

В старых версиях Python порядок разрешения методов был достаточно примитивным: поиск вёлся во всех родительских классах слева направо на максимальную глубину. Т.е. если у родительского класса в свою очередь не было нужного метода, но были родители, то поиск производился в них по тем же правилам. Рассмотрим структуру:

```
1 A     C
2 |     |
3 B     D
4  \   /
5    E
```

При обращении к методу экземпляра класса E такой алгоритм произвёл бы поиск последовательно в классах E, B, A, D и C. Таким образом поиск вёлся сначала в первом классе-родителе и во всех его предках, затем во втором классе-родителе со всеми предками и т. д.

Однако, начиная с версии 2.2, появился базовый класс object, от которого рекомендовалось наследовать все пользовательские классы. Начиная с версии 3.0, старые классы больше не поддерживаются, а все пользовательские классы по умолчанию происходят от класса object.

Это породило проблему «ромбовидной структуры» («diamond diagram»).

```
1   object
2    /   \
3   A     B
4    \   /
5      C
```

Если у нас есть классы A и B, от которых наследуется класс C, то при поиске метода по старому алгоритму (C, A, object, B) получается, что если метод не определён в классах C и A он будет извлечён из object, даже если он определён в B. Это создаёт определённые неудобства, т.к. в object определены многие магические методы, типа `__init__`, `__str__` и т.п. Но даже если object заменить на некий пользовательский класс D, то проблема останется — менее специфичный метод класса-предка может отработать вместо более специфичного метода класса-потомка.

Итак, у нас на руках есть сам класс, список всех его предков и связей между ними. Из этих данных нам нужно построить упорядоченный список классов, в которых будет производиться поиск метода слева направо. Такой список называется линеаризацией класса. Для простоты возьмем структуру:

```
1 object
2    |
3    A
4    |
```

```
5    B
```

Тогда линеаризацией для класса B будет список `[B, A, object]`. Т.е. при вызове B().something() метод сначала будет искаться в классе B. Если он там не найден, то поиск продолжится в классе A. Если его нет и там, то поиск завершится в классе object. Перебрав все классы из линеаризации и не обнаружив нужного метода, Питон выкинет ошибку Attribute Error.

Для решения проблемы ромбовидной структуры линеаризация должна быть монотонной. Это значит, что если в линеаризации некого класса C класс A следует за классом B (она имеет вид `[C, …, B, …, A]`), то и для любого его потомка D класс B будет следовать за A в его линеаризации (она будет иметь вид `[D, …, C, …, B, …, A]`). Как можно заметить, старый порядок разрешения методов не монотонен, т.к. в случае ромбовидной структуры для класса A линеаризация есть `[A, object]`, для класса B — `[B, object]`, но для класса C — `[C, A, object, B]`, что нарушает свойство монотонности по отношению к классу B.

Для удовлетворения свойству монотонности в этом случае подойдут две линеаризации: `[C, A, B, object]` и `[C, B, A, object]`. Очевидно, что они обе не нарушают монотонности ни в отношении класса A (т.к. object следует за A в обоих случаях), ни в отношении класса B (т.к. object следует за B в обоих случаях). Так какую из них выбрать? В этом случае наиболее интуитивно-понятный способ — посмотреть на определение класса C. Если класс объявлен как C(A, B), то разумно будет взять первую линеаризацию, поскольку в ней B следует за A. Если класс объявлен как C(B, A), то будет лучше взять вторую линеаризацию, в которой A следует за B.

Такой выбор определяется порядком локального старшинства (local precedence ordering). Свойство порядка локального старшинства требует соблюдения для классов-родителей в линеаризации класса-потомка того же порядка, что и при его объявлении. Например, если класс объявлен как D(A, B, C), то в линеаризации D класс A должен стоять раньше B, а класс B — раньше C.

Алгоритм порядка разрешения методов C3

Для достижения указанных выше целей в Python используется алгоритм C3. Он достаточно прост, однако для лучшего понимания введём следующие условные обозначения: `[C1, C2, … CN]` – список из элементов `C1, C2, … CN`. Соответственно, `[C]` — список из одного элемента `C`. `L[C]` – линеаризация класса C. Важно помнить, что любая линеаризация есть список по определению. `merge(L[C1], L[C2], …, L[CN])` – объединение элементов линеаризаций `L[C1], L[C2], …, L[CN]` в список с помощью некоторого алгоритма. По сути, этот алгоритм должен упорядочить все классы из `L[C1], L[C2], …, L[CN]` и исключить дублирование классов в итоговом списке.

Алгоритм C3 представляет из себя набор следующих правил:

- Линеаризация класса C есть одноэлементный список из самого класса `C` плюс объединение линеаризаций его родителей и списка всех его родителей. В условных обозначениях это можно записать как `L[C] = [C] + merge(L[C1], L[C2], …, L[CN], [C1, C2, …, CN])`, если класс C был объявлен как `class C(C1, C2, …, CN)`. Надо отметить, что каждая линеаризация `L[CX]` начинается с класса `CX`, который был дополнительно приписан в конец списка объединения как непосредственный родитель класса `C`. Зачем это сделано станет ясно далее.
- Объединение линеаризаций происходит следующим образом:
  1. Берётся нулевой элемент из первой линеаризации `(L[C1][0])`.

2. Этот элемент ищется во всех других линеаризациях (от `L[C2]` до `L[CN]`).
3. Если этот элемент найден где-то вне начала списка (`L[CK][X] == L[C1][0], X != 0`; по сути это значит, что `L[C1][0]` является чьим-то предком), то алгоритм переходит к первому шагу, беря в качестве нулевого элемент из следующей линеаризации (`L[C2][0]`).
4. Если элемент нигде не найден в позиции отличной от нулевой, то он добавляется в конец итогового списка линеаризации и удаляется из всех рассматриваемых списков (от `L[C1]` до `L[CN]`; один и тот же класс не может встречаться в итоговом списке дважды). Если после удаления элемента остались пустые списки — они исключаются из объединения. После этого алгоритм повторяется с самого начала (от нового элемента `L[C1][0]`), если он есть. Если его нет — объединение закончено.
5. Если алгоритм дошёл до последнего элемента `L[CN]`, но ни один из нулевых элементов не удовлетворяет правилам, то линеаризация не возможна.

Чтобы алгоритм стал понятнее, разберём несколько примеров. Первым делом посмотрим, что же теперь происходит с нашей ромбовидной структурой. Для неё мы получаем: `L[C] = [C] + merge(L[A], L[B], [A, B])` `L[A] = [A] + merge(L[object], [object])` `L[B] = [B] + merge(L[object], [object])` `L[object] = [object]` (вырожденный случай)

Наибольший интерес представляет процесс объединения, так что разберём его более подробно. В случаях `L[A]` и `L[B]` выражение `merge(L[object], [object]) = merge([object], [object])` раскрывается тривиально. Поскольку и первый и второй списки состоят из одного элемента `object`, по пункту 4 правила объединения результатом будет `[object]`.

Теперь разберём `L[C] = [C] + merge(L[A], L[B], [A, B]) = [C] + merge([A, object], [B, object], [A, B])`. Распишем наше объединение по правилам C3:

- Возьмем нулевой элемент первого списка — `A`.
- Поищем его во всех прочих списках, т.е. в `[B, object]` и `[A, B]`.
- Поскольку класс `A` не обнаружен в списке `[B, object]`, а в списке `[A, B]` является нулевым элементом, то мы добавляем его в итоговый список линеаризации `L = [] + [A] = [A]`. После этого класс `A` нужно удалить изо всех списков в объединении. Получим `L[C] = [C] + [A] + merge([object], [B, object], [B])`.
- Снова возьмём нулевой элемент первого списка — `object`.
- Поищем его во всех других списках и обнаружим, что object является первым (не нулевым) элементом списка `[B, object]`. Как уже было сказано выше, по сути, это означает, что класс object предок класса B. Стало быть, сначала имеет смысл поискать более специфичный метод в классе B. Поэтому алгоритм справедливо возвращается к пункту 1.
- Снова возьмём нулевой элемент, но на этот раз уже из второго списка, т.е. `B` из списка `[B, object]`.
- Поищем его в других списках и обнаружим его только в третьем списке `[B]` в нулевой позиции, что нас вполне устраивает. Следовательно, добавим его в итоговый список, а затем удалим изо всех списков в объединении. Получим `L = [A] + [B] = [A, B]` и соответственно `L[C] = [C] + [A, B] + merge([object], [object])`.
- Полученное объединение `merge([object], [object])` уже рассматривалось выше. В итоге получим `L[C] = [C] + [A, B] + [object] = [C] + [A, B, object] = [C, A, B, object]`. Список `[A, B, object]` — это и есть результат нашего объединения `merge(L[A], L[B], [A, B])`.

При создании класса (во время выполнения инструкции `class`) интерпретатор автоматически рассчитывает линеаризацию с помощью алгоритма C3 и сохраняет её в неизменяемый кортеж в атрибуте класса `__mro__` (или `Class.mro()`).

Когда происходит вызов `instance.method()`, Python не выполняет алгоритм C3 заново. Он просто последовательно проходит по готовому кортежу `type(instance).__mro__` и берёт первый найденный метод.

Если вы хотите получить линеаризацию для самопроверки или каких-то других целей — используйте свойство класса `__mro__` (например, `C.__mro__`).

Функция `super()` работает не с «родительским классом», а **строго по порядку в MRO текущего экземпляра**. Именно поэтому в множественном наследовании `super().method()` передаёт управление «соседу» справа по цепочке MRO (кооперативный вызов методов).

Виртуальное наследование (из C++) не может быть включено в **MRO** (Method Resolution Order), потому что MRO строит **линейную** последовательность классов, тогда как виртуальное наследование создает **нелинейную** структуру данных в памяти (граф с разделяемым базовым подобъектом).

Давайте наглядно посмотрим на пример поиска метода:

```python
def something_else(place = "object"):
  print(f"Hi from {place}")

class A:
   def something(self = object):
     print("Hi from superclass")

class B(A):
    def __init__(self):
       self.something = something_else
    def something(self = object):
        print("Hi from class")

B().something()
```

```
Hi from object
```

```python
B.something()
```

```
Hi from class
```

```python

def also_something_else(place="code"):
    something_else(place)

b = B()
b.something = also_something_else
b.something()
```

```
Hi from code
```

```python
del b.something
b.something()
```

```
Hi from class
```

```python
del B.something
b.something()
```

```
Hi from superclass
```

Так что алгоритм дескриптора примерно следующий:

Поиск в неймспейсе объекта -> в неймспейсе класса -> неймспейсе предка

Инкапсуляция

Сокрытие

При проектировании языка python исходили из концепции: “все мы взрослые люди”.
Поэтому в python нет классических `public`, `protect` и `private` полей.

Однако вместо этого есть соглашение о наименовании доступов. И PEP 8 они описываются так:

- `_single_leading_underscore`: слабый индикатор “внутреннего использования”. Например, `from M import *` не импортирует объекты, имена которых начинаются с подчеркивания.
- `single_trailing_underscore_`: используется по соглашению во избежание конфликтов с ключевым словом Python, например:
- `tkinter.Toplevel(master, class_='ClassName')`
- `__double_leading_underscore`: при именовании атрибута класса вызывает искажение имени (внутри класса FooBar `__boo` становится `_FooBar__boo`; см. ниже).
- `__double_leading_and_trailing_underscore__`: «магические» объекты или атрибуты, которые находятся в управляемых пользователем пространствах имен. Например, `__init__`, `__import__` или `__file__`. Никогда не придумывайте такие имена; используйте их только в соответствии с документацией.

Полиморфизм

Перегрузка

В python в отличии от C++ нет классической перегрузки.
В том числе потому что часть сути

```python
def func(a, b):
    return a+b

func(1,2)
```

```
3
```

```python
func("1", "2")
```

```
'12'
```

Утиная типизация

Основная идея утиной типизации - это принцип “если что-то выглядит как утка, крякает утка, то это утка” и python переполнен примерами применения

Protocol и Interface

Метаклассы

По сути метаклассы - это классы объектами которами являются другие классы.
Раньше мы применяли type как функцию, которая принимает объект и возвращает класс объекта.
Но на самом деле это класс, который при передаче одного аргумента возвращает класс, к которому относится объект

```python
class MySuperClass: ...
class MyMixin: ...

class MyClass(MySuperClass, MyMixin):
    x = 42
    def x2(self):
        return self.x * 2

MyClass()
```

```
<__main__.MyClass object at 0x10100b0e0>
```

С помощью конструктора type мы можем создать MyClass во время выполнения:

```python
MyClass = type('MyClass',
        (MySuperClass, MyMixin),
        {'x': 42, 'x2': lambda self: self.x * 2},
    )

MyClass()
```

```
<__main__.MyClass object at 0x10100b0e0>
```